

Network Intrusion Detection Using Machine Learning (ML)

From Artificial intelligence(AI)/ML basics to hands-on cybersecurity experiments

Muhammad Syafrudin
Kookmin University, Korea



Applied INtelligence (AIN) Lab • ain.kookmin.ac.kr

Learning Outcomes

By the end, students can:

- Explain the difference between **AI, machine learning**, and **classification**
- Describe **network intrusion detection** as a **binary classification** task
- Load and inspect a tabular cybersecurity dataset
- Build a **preprocessing and model pipeline** with scikit-learn
- Compare models using **precision, recall, F1-score, ROC-AUC**, and a confusion matrix
- Save a model with `joblib` and use it to predict a CSV file

200-Minute Lesson Flow

Time	Focus	Activity
0-20	AI and ML concepts	Guided introduction
20-40	Cybersecurity context	What is network intrusion detection?
40-65	Dataset exploration	pandas inspection
65-95	Preprocessing	encoding and scaling
95-130	Modeling	train three classifiers
130-160	Evaluation	metrics and confusion matrix
160-180	Model saving	<code>joblib</code> pipeline

What Is AI?

Artificial Intelligence (AI) is the broad idea of machines performing tasks that normally require human intelligence.

Examples:

- Recognizing suspicious network behavior
- Translating text
- Recommending videos
- Detecting fraud
- Planning robot movement

What Is ML?

Machine Learning (ML) is a way to build AI systems by learning patterns from data.

Instead of writing every rule manually:

```
if traffic is weird then alert
```

we train from examples:

```
features + known label → learned model
```

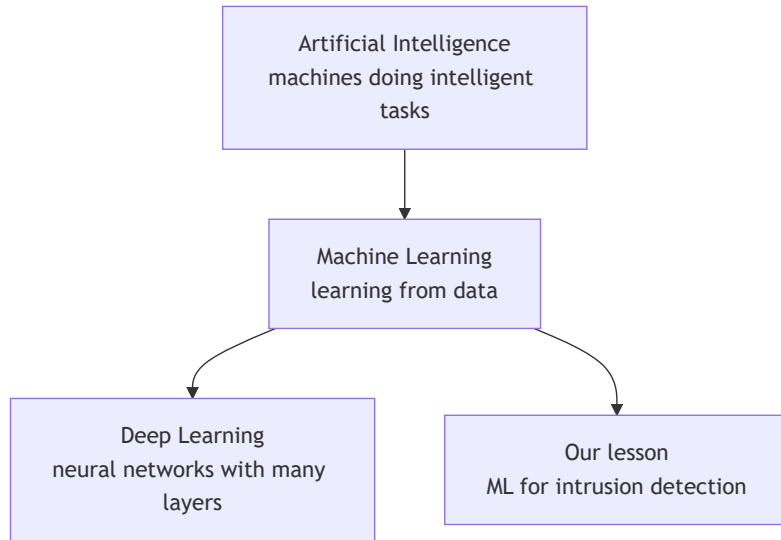
Machine Learning Workflow



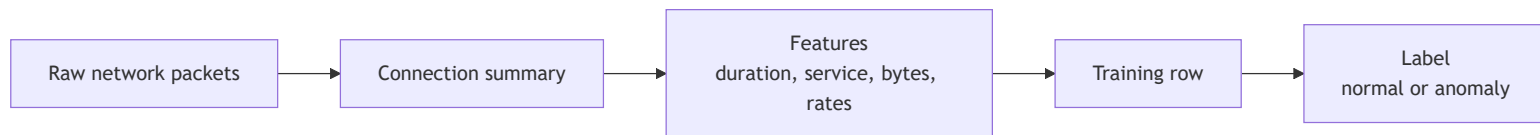
Key idea: the model should perform well on **new data**, not only on rows it already saw.

AI, ML, and Deep Learning

Use this figure to position today's work: we are not covering all AI. We are learning one practical **supervised ML workflow**.



From Network Traffic to a Dataset



The model does not inspect every raw packet directly in this lesson. It learns from engineered **connection-level features**.

Supervised Classification

This lesson uses **supervised learning**.

- We have **input features**: connection duration, protocol, service, byte counts, error rates, and more
- We have a known **training label**: `normal` or `anomaly`
- The model learns to predict the **label** for future connections

`network connection features → ML model → normal or anomaly`

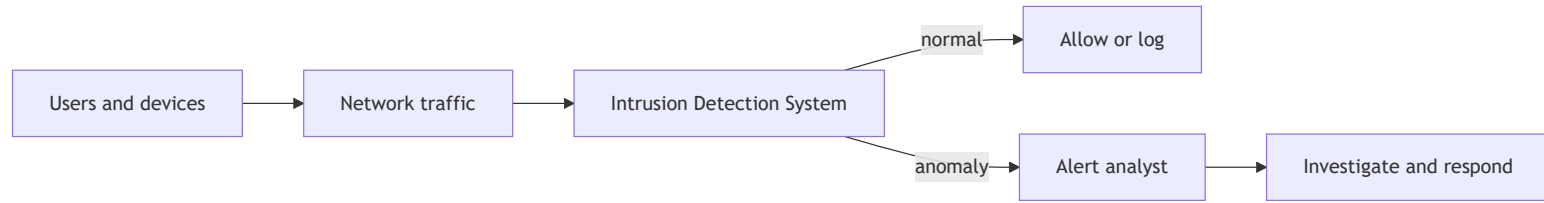
Cybersecurity Context

A **network intrusion detection system (NIDS)** monitors network traffic and raises alerts when behavior looks suspicious.

Examples of suspicious behavior:

- Repeated failed connection attempts
- Unusual services or protocols
- Abnormal error rates
- Strange byte patterns
- Activity that differs from normal usage

Intrusion Detection View



Important teaching point: an **ML model** is only one component of a larger **security process**.

The Dataset

Files in this project:

- `dataset/Train_data.csv` : **labeled training data**
- `dataset/Test_data.csv` : **unlabeled data** for prediction demo

Each row is one network connection.

The **target column** in the training set is `class` :

- `normal`
- `anomaly`

Dataset Shape as a Figure

Train_data.csv

Rows: network connections

Columns: 41 features + 1 label

duration	protocol_type	service	...	class
0	tcp	http	...	normal
0	tcp	private	...	anomaly

Each row becomes one **example** for supervised learning.

Feature Types

Categorical features:

- `protocol_type`
- `service`
- `flag`

Numerical features:

- `duration`
- `src_bytes`
- `dst_bytes`
- `count`
- error rates and host rates

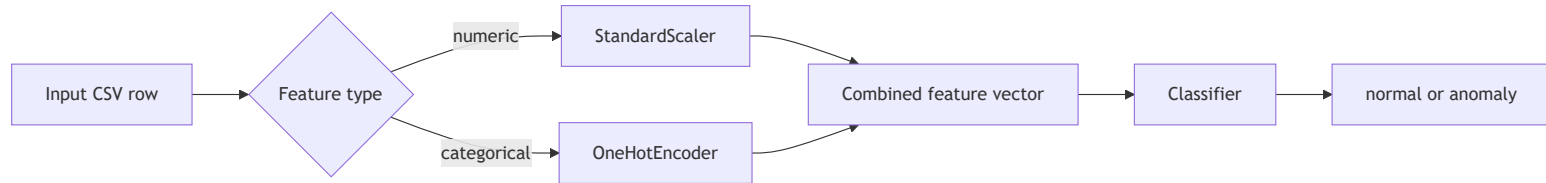
Why Preprocess?

Most **ML algorithms** need numbers.

We use:

- `OneHotEncoder` for categories
- `StandardScaler` for numeric values
- `ColumnTransformer` to apply both correctly
- `Pipeline` so training and prediction use the same steps

Preprocessing Pipeline Figure



The same **fitted pipeline** must be reused for training, validation, and future CSV prediction.

Notebook Setup

```
from pathlib import Path
import joblib
import pandas as pd

TRAIN_PATH = Path("dataset/Train_data.csv")
TEST_PATH = Path("dataset/Test_data.csv")
MODEL_PATH = Path("models/network_intrusion_pipeline.joblib")
```

This keeps the notebook **portable** inside the project folder.

Load and Inspect Data

```
train_df = pd.read_csv(TRAIN_PATH)
test_df = pd.read_csv(TEST_PATH)

print(train_df.shape)
train_df.head()
train_df["class"].value_counts()
```

Questions for students:

- How many rows are available?
- Are labels balanced?
- Which columns are categorical?
- Does the test file include labels?

Train/Validation Split

```
X = train_df.drop(columns=["class"])
y = train_df["class"].map({"normal": 0, "anomaly": 1})

X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)
```

Why split data?

- Train on one part
- Test on another part
- Estimate performance on future unseen data

Preprocessing Pipeline

```
preprocessor = ColumnTransformer(  
    transformers=[  
        ("numeric", StandardScaler(), numeric_features),  
        ("categorical", OneHotEncoder(handle_unknown="ignore"), categorical_features)  
    ]  
)
```

`handle_unknown="ignore"` helps when a future CSV contains a category not seen during training.

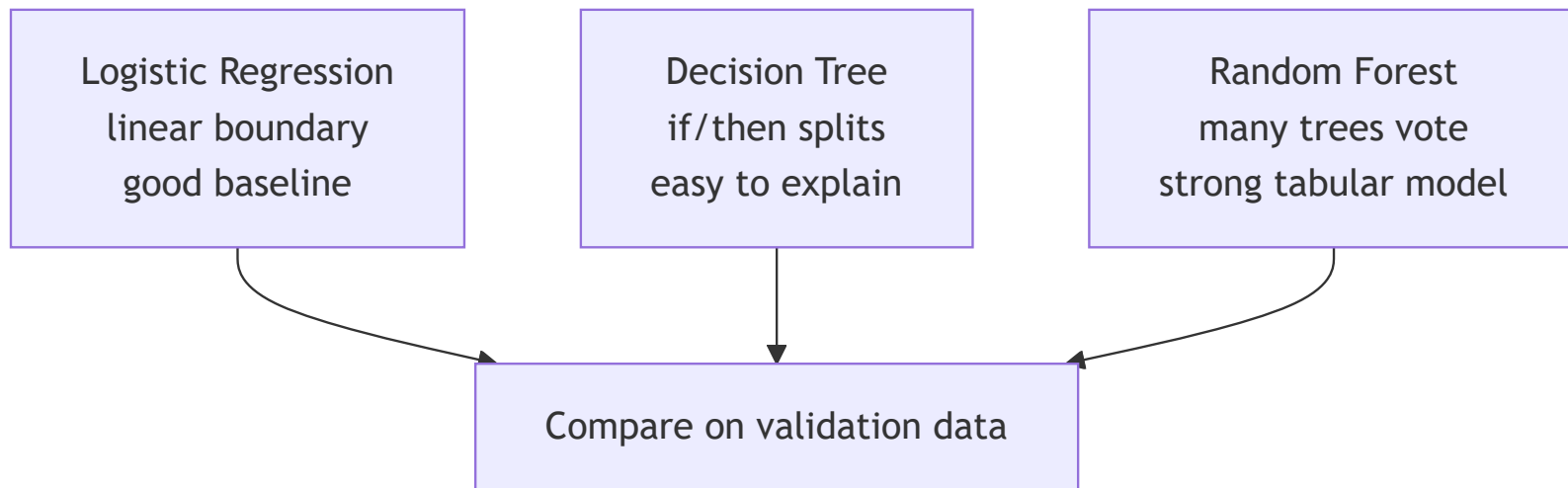
Models to Compare

We compare three beginner-friendly **classifiers**:

- **Logistic Regression**: simple baseline
- **Decision Tree**: visual and explainable
- **Random Forest**: stronger tabular model using many trees

The notebook ranks models by **anomaly F1-score**.

Model Intuition



The goal is not to memorize algorithms. The goal is to compare evidence from **evaluation metrics**.

Evaluation Metrics

Accuracy alone can be misleading in security.

Important metrics:

- **Precision:** when the model predicts anomaly, how often is it right?
- **Recall:** how many real anomalies did it catch?
- **F1-score:** balance between precision and recall
- **ROC-AUC:** ranking quality across thresholds
- **Confusion matrix:** shows false alarms and missed attacks

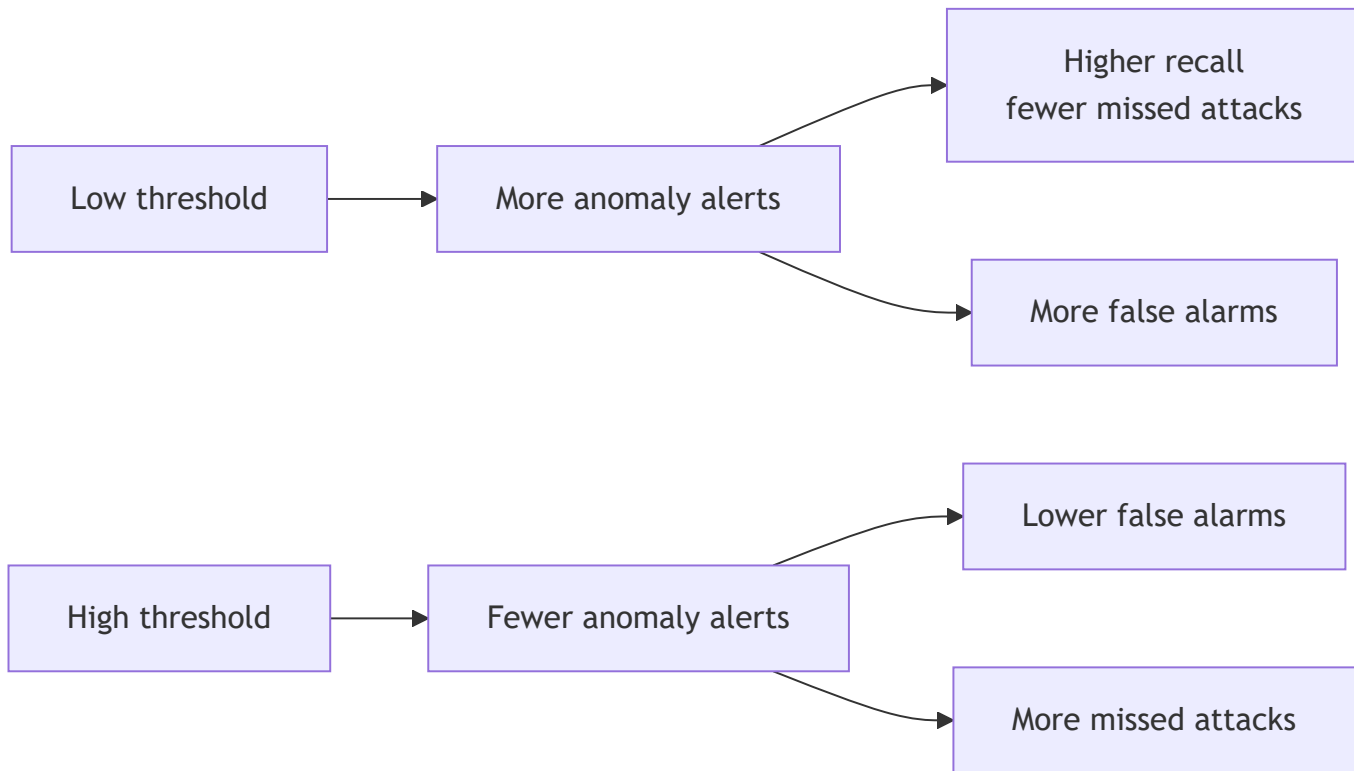
Confusion Matrix

	Predicted normal	Predicted anomaly
Actual normal	true normal	false alarm
Actual anomaly	missed attack	detected attack

Discussion:

- A **missed attack** can be dangerous
- Too many **false alarms** can exhaust security teams
- Real systems must **balance both risks**

Metric Trade-Off Illustration



Save the Model

```
joblib.dump(best_pipeline, MODEL_PATH)
```

We save the **whole pipeline**, not only the classifier.

Why?

- The model needs the same **encoding** at prediction time
- The model needs the same **scaling** at prediction time
- It reduces mistakes when using **new CSV files**

CSV Prediction Demo

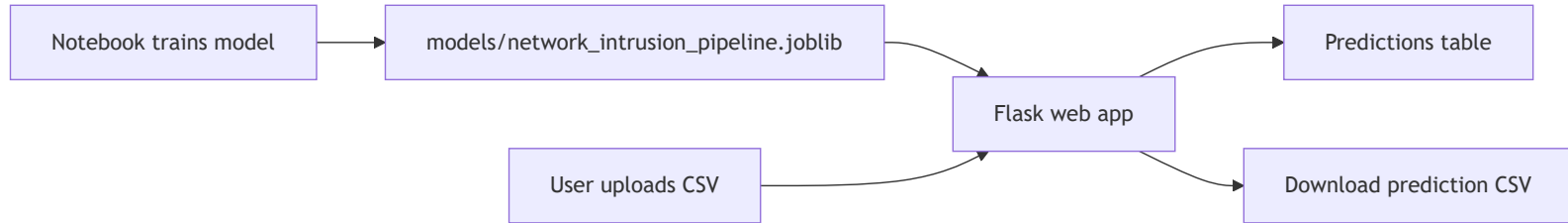
```
loaded_model = joblib.load(MODEL_PATH)
new_df = pd.read_csv("dataset/Test_data.csv")
predictions = loaded_model.predict(new_df)
```

The notebook wraps this into:

```
analyze_csv_with_model(csv_path, model_path, output_path)
```

It writes a CSV containing `predicted_class` and **anomaly probability**.

End-to-End Demo Architecture

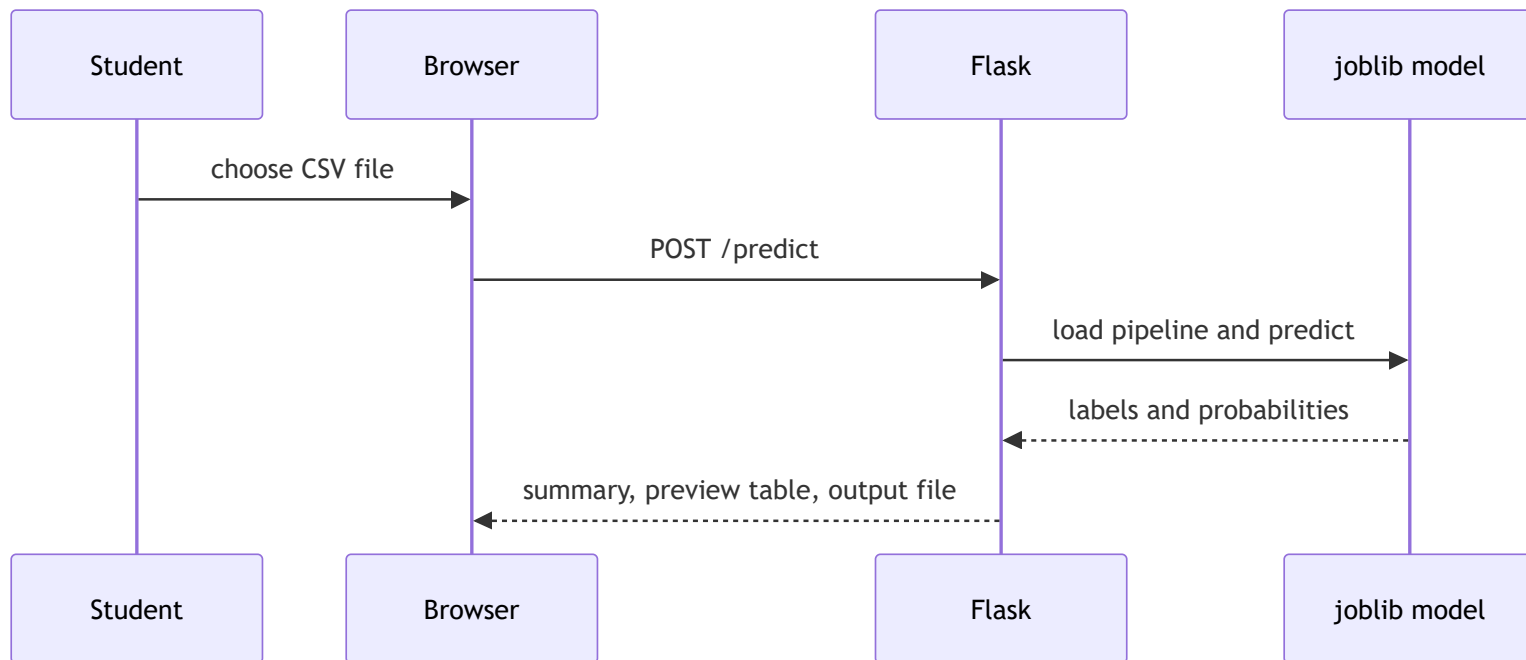


This final demo turns the experiment into a small **usable application**.

Web Demo Interface

Prediction Result Interface

Flask Demo Flow



Run after the notebook has saved `models/network_intrusion_pipeline.joblib`.

Final Demo Commands

```
pip install -r requirements.txt
jupyter notebook network-intrusion-detection-using-ml.ipynb
python webapp\app.py
```

Then open:

```
http://127.0.0.1:5000
```

Upload dataset/Test_data.csv ,
outputs/sample_network_connections.csv , or
outputs/web_demo_upload.csv .

Hands-On Student Tasks

During the lab, students should:

- Run each notebook section in order
- Identify the **target** and **feature columns**
- Compare at least three **ML models**
- Explain which model they would choose and why
- Use the saved `joblib` **model** on `dataset/Test_data.csv`
- Launch the **Flask web app** and upload a CSV file
- Open the generated **prediction CSV** and summarize the result

Ethics and Limitations

ML intrusion detection is helpful, but imperfect.

- Models can miss **new attack types**
- Models can create **false alarms**
- Training data may not match a real organization's traffic
- Attackers may adapt after learning detection patterns
- **Human review** and **security policy** are still required

Wrap-Up

Today we built a complete applied ML workflow:

data -> preprocessing -> training -> evaluation -> saved model -> CSV prediction

Next step: **tune the model** and test it with more realistic network traffic.



Applied INtelligence (AIN) Lab • ain.my.id